Name : CHANDRASHEKHARA K M

GITHUB: https://github.com/ChandrashekharaKM

Part 2: Explanation

I structured the app around two feature areas dashboard and product form with a thin core layer holding the two Firestore-facing services and the auth service. For a below-medium-scope task with one collection pair and five features, that's enough separation: it keeps Firestore access out of components without introducing a state-management layer the app doesn't need yet.

Stock adjustments go through a Firestore transaction rather than a plain update, so the quantity change and the movement-log write succeed or fail together. That was the one piece of "extra" design beyond the brief, because a dashboard that silently loses a movement record under concurrent edits felt like the kind of bug that's invisible until it isn't.

If this needed to handle 10,000 users a day, the biggest change would be data shape: right now every client subscribes to the full products collection. At real scale I'd paginate the dashboard query, add a composite index for the low-stock filter instead of filtering client-side, and move the stock-adjustment transaction into a Cloud Function so business rules (e.g., blocking negative stock, rate-limiting adjustments) aren't only enforced client-side and in security rules. I'd also add Firestore usage monitoring and budget alerts, since **`collectionData`** subscriptions can get expensive at scale if not scoped per shop.

One thing I left out given the time constraint: multi-user/multi-shop scoping. Right now all signed-in users see the same product set. Given more time I'd add a **`shopId`** on products and movements, scope every query to the signed-in user's shop, and update the security rules to check **`resource.data.shopId == request.auth.token.shopId`** via a custom claim.

Beyond the core stack, I used Angular's Signals (**`toSignal, computed)`** for the search/filter state instead of RxJS operators chained in the template, mainly for readability — it keeps the filtering logic in the component class as plain, testable functions rather than a stream pipeline that's harder to reason about at a glance.